```
var corsOptions = {
  allowOrigin: createWhitelistValidator(originWhitelist),
  allowCredentials: true,
  shortCircuit: true,
  allowMethods: ['GET', 'DELETE'],
  allowHeaders: ['Timezone-Offset'],
  maxAge: 120,
  exposeHeaders: ['X-Powered-By']
};

...

serverapp.get('/api/posts', function(req, res) {
  res.json(POSTS);
});
```

Allows Timezone-Offset request header, but not Sample-Source header

Removes redirect from chapter 6

These configuration options are essentially the same as those from the previous chapter, but with one critical difference. This code only allows the Timezone-Offset request header, even though the client is configured to send both the Timezone-Offset and the Sample-Source request headers. This means that the preflight request will fail, because the Access-Control-Allow-Headers response header won't match the Access-Control-Request-Headers request header (figure 7.2).

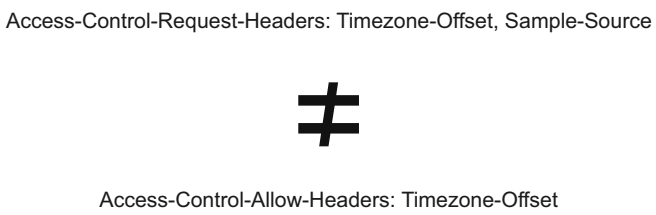


Figure 7.2 The CORS request fails because the Access-Control-Request-Headers doesn't match the Access-Control-Allow-Headers header.

The fix to this error is as easy as adding the Sample-Source header to the `allowHeaders` array. But even though you know how to fix this, set this knowledge aside and forget about it for a while. The rest of this chapter will approach this sample as if you were debugging an unknown error, and will walk through the tools that can be used to debug the request.

7.2 Using the browser's developer tools

If you restart the sample server and visit the client page, you'll be greeted with the error page rather than the blog posts. To figure out what went wrong, let's start by inspecting the error using your browser's developer tools.

All modern browsers have a suite of developer tools, which are in-browser panels that can help a web developer catch and diagnose issues with a site. The following sidebar explains how to find the developer tools in various browsers.